

Comandos SQL – *Structured Query Language* – no SGBD PostgreSQL

1. Download e Instalação: <https://www.postgresql.org/download/>
2. Documentação: <https://www.postgresql.org/docs/>
3. Nesta apostila, os exemplos com gerenciamento de banco de dados serão demonstrados no sistema operacional Ubuntu 22.04 com o cliente de linha de comando **psql**: <https://www.postgresql.org/docs/current/app-psql.html>
4. Após instalação, para logar no **psql** com o usuário **postgres**, será pedida a senha de seu sistema operacional: **sudo -u postgres psql**
5. Para definir um usuário com senha e deixar o acesso mais seguro às bases de dados: <https://www.youtube.com/watch?v=HbAJQ1WuGRE>
6. No **psql**, veja a lista de comandos disponíveis com **\?** e depois para voltar à linha de comando do **psql** basta digitar **q**;
 - **\du** lista os usuários definidos;
 - **\l** lista as bases de dados existentes;
 - Use **\c** para se conectar a alguma base de dados existente;
7. Criando uma base de dados **colegio** com o comando **SQL CREATE DATABASE:**

criar uma base de dados.
o Database name.

```
postgres=# CREATE DATABASE/colégio;
CREATE DATABASE
postgres=# \l

  Nome      | Dono      | Codificação | U
-----+-----+-----+--
 aula       | postgres  | UTF8        | T
 colegio    | postgres  | UTF8        | T
 postgres   | postgres  | UTF8        | T
 template0  | postgres  | UTF8        | T
 template1  | postgres  | UTF8        | T
(5 registros)

postgres=#
```

8. Após usar **\c** para se conectar na base de dados **colegio**, pode-se listar todas as tabelas existentes com **\dt.** ou então criar tabelas com o comando SQL **CREATE TABLE**. Vamos criar uma tabela para os alunos. Observe as configurações dos campos como sequência automática (SERIAL), chave primária, não nulo e os tipos de dados. Utilize **\d** para ver os detalhes da tabela criada:

```

colégio=# \dt
Did not find any relations.
colégio=# CREATE TABLE alunos ( id SERIAL PRIMARY KEY, nome TEXT NOT NULL, idade INT);
CREATE TABLE
colégio=# \d alunos;

```

Coluna	Tipo	Collation	Nullable	Default
id	integer		not null	nextval('alunos_id_seq'::regclass)
nome	text		not null	
idade	integer			

```

Índices:
    "alunos_pkey" PRIMARY KEY, btree (id)
colégio=#

```

9. Vamos inserir registros na tabela alunos com o comando SQL **INSERT INTO**:

```

colégio=# INSERT INTO alunos (nome, idade) VALUES ('João', 29);
INSERT 0 1
colégio=# INSERT INTO alunos (nome, idade) VALUES ('Maria', 30);
INSERT 0 1
colégio=# INSERT INTO alunos (nome, idade) VALUES ('Tereza', 25);
INSERT 0 1
colégio=#

```

10. Listando os registros com o comando SQL **SELECT**:

```

colégio=# SELECT * FROM alunos;
 id | nome  | idade
----+-----+-----
  1 | João  |    29
  2 | Maria |    30
  3 | Tereza |    25
(3 registros)
colégio=#

```

11. Após inserir mais alguns registros, vamos atualizar com o comando SQL **UPDATE** o nome do Higo (está escrito errado com y) e sua idade. Cuidado para não esquecer a cláusula **WHERE**, pois através dela sabe-se qual(is) registro(s) atualizar com a inclusão de operações relacionais, lógicas e aritméticas:

```
colegio=# SELECT * FROM alunos;
```

id	nome	idade
1	João	29
2	Maria	30
3	Tereza	25
4	Carlos	20
5	Hygo	20

(5 registros)

```
colegio=# UPDATE alunos SET nome = 'Higo', idade = 31 WHERE id = 5;
```

```
UPDATE 1
```

```
colegio=# SELECT * FROM alunos;
```

id	nome	idade
1	João	29
2	Maria	30
3	Tereza	25
4	Carlos	20
5	Higo	31

(5 registros)

```
colegio=#
```

12. Agora vamos completar o CRUD – *Create Read Update Delete* – com o comando SQL **DELETE**. Será removido o registro do aluno Carlos. Também é necessário a cláusula **WHERE** para se saber qual(is) registro(s) a deletar:

```
colegio=# DELETE FROM alunos WHERE id = 4;
```

```
DELETE 1
```

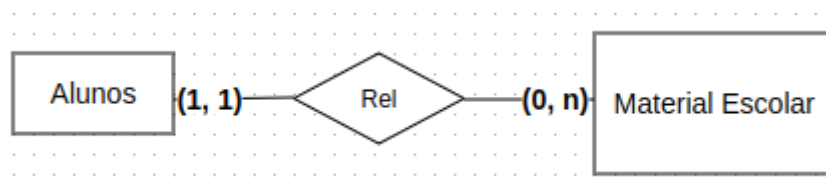
```
colegio=# SELECT * FROM alunos;
```

id	nome	idade
1	João	29
2	Maria	30
3	Tereza	25
5	Higo	31

(4 registros)

```
colegio=#
```

13. Agora vamos criar outras tabelas e trabalhar os conceitos de relacionamentos e suas cardinalidades. Será criada a tabela **material_escolar** para demonstrar um relacionamento de 1 para N, onde cada aluno poderá ter muitos materiais escolares:



```

colégio=# CREATE TABLE material_escolar ( id SERIAL PRIMARY KEY, descricao TEXT NOT NULL, valor MONEY, id_aluno INT REFERENCES alunos (id));
CREATE TABLE
colégio=# \d material_escolar;
          Tabela "public.material_escolar"
  Coluna | Tipo | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id      | integer |          | not null | nextval('material_escolar_id_seq'::regclass)
 descricao | text   |          | not null |
 valor   | money  |          |          |
 id_aluno | integer |          |          |
Índices:
 "material_escolar_pkey" PRIMARY KEY, btree (id)
Restrições de chave estrangeira:
 "material_escolar_id_aluno_fkey" FOREIGN KEY (id_aluno) REFERENCES alunos(id)
colégio=#

```

Note o uso de **REFERENCES** para designar a chave estrangeira apontando para o campo **id** da tabela **alunos**. Crie alguns registros para esta tabela e perceba que para não ferir a integridade de chave estrangeira, no momento da inserção, ao preencher um valor para a chave estrangeira **id_aluno**, já deve existir no campo **id** da tabela **alunos**:

```

colégio=# INSERT INTO material_escolar (descricao, valor, id_aluno) VALUES ('caneta azul', 1.56, 1);
INSERT 0 1
colégio=# INSERT INTO material_escolar (descricao, valor, id_aluno) VALUES ('caneta vermelha', 1.90, 1);
INSERT 0 1
colégio=# INSERT INTO material_escolar (descricao, valor, id_aluno) VALUES ('caneta preta', 1.50, 2);
INSERT 0 1
colégio=# INSERT INTO material_escolar (descricao, valor, id_aluno) VALUES ('caneta preta', 1.50, 99);
ERRO:  inserção ou atualização em tabela "material_escolar" viola restrição de chave estrangeira "material_escolar_id_aluno_fkey"
DETALHE:  Chave (id_aluno)=(99) não está presente na tabela "alunos".
colégio=# select * from material_escolar;
 id | descricao | valor | id_aluno
----+-----+-----+-----
  1 | caneta azul | R$ 1,56 |      1
  2 | caneta vermelha | R$ 1,90 |      1
  3 | caneta preta | R$ 1,50 |      2
(3 registros)
colégio=#

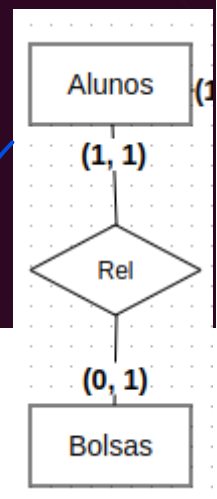
```

14. Agora um exemplo de um relacionamento 1 para 1. Será criada a tabela **bolsas** e cada aluno poderá ter apenas uma bolsa e a mesma ser para somente um aluno. Note a chave estrangeira **id_aluno** sendo também uma chave primária para garantir esse relacionamento e que não pode ser sequencial (SERIAL):

```

colégio=# CREATE TABLE bolsas ( id_aluno INT PRIMARY KEY REFERENCES alunos (id), tipo TEXT, valor MONEY);
CREATE TABLE
colégio=# \d bolsas;
          Tabela "public.bolsas"
  Coluna | Tipo | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id_aluno | integer |          | not null |
 tipo     | text   |          |          |
 valor    | money  |          |          |
Índices:
 "bolsas_pkey" PRIMARY KEY, btree (id_aluno)
Restrições de chave estrangeira:
 "bolsas_id_aluno_fkey" FOREIGN KEY (id_aluno) REFERENCES alunos(id)
colégio=#

```



Note que ao inserir os registros nessa tabela, deve ser atendido também as integridades de chaves primária e estrangeira:

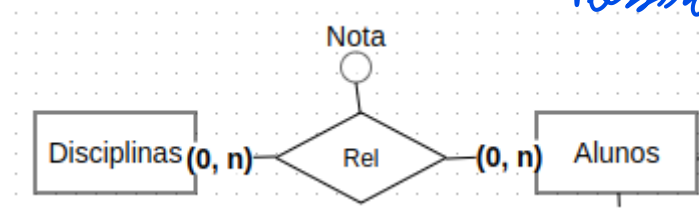
```

colegio=# INSERT INTO bolsas (id_aluno, tipo, valor) VALUES (2, 'científica', 700.00);
INSERT 0 1
colegio=# INSERT INTO bolsas (id_aluno, tipo, valor) VALUES (3, 'monitoria', 400.00);
INSERT 0 1
colegio=# INSERT INTO bolsas (id_aluno, tipo, valor) VALUES (3, 'estagiário', 1000.00);
ERRO: duplicar valor da chave viola a restrição de unicidade "bolsas_pkey"
DETALHE: Chave (id_aluno)=(3) já existe.
colegio=# select * from bolsas;
 id_aluno |  tipo   |  valor
-----+-----+-----
       2 | científica | R$ 700,00
       3 | monitoria | R$ 400,00
(2 registros)

colegio=#

```

15. Agora para fechar os relacionamentos e suas cardinalidades, um exemplo de relacionamento N para N, onde cada aluno poderá ter muitas disciplinas e cada disciplina ser composta por vários alunos. Serão criadas as tabelas **disciplinas** e **alunos_disciplinas**, sendo essa última a que possibilita esse relacionamento N para N no modelo relacional, através da combinação de chaves estrangeiras como chave primária composta:



1 disciplina ~ 0 ~

```

colegio=# CREATE TABLE disciplinas ( id SERIAL PRIMARY KEY, nome TEXT, ch INT);
CREATE TABLE
colegio=# \d disciplinas;
          Tabela "public.disciplinas"
Coluna | Tipo | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id     | integer |           | not null | nextval('disciplinas_id_seq'::regclass)
 nome   | text    |           |          |
 ch     | integer |           |          |
Índices:
 "disciplinas_pkey" PRIMARY KEY, btree (id)

```

```

colegio=# CREATE TABLE alunos_disciplinas ( id_aluno INT REFERENCES alunos (id), id_disciplina INT REFERENCES disciplinas (id), nota REAL, PRIMARY KEY (id_aluno, id_disciplina));
CREATE TABLE
colegio=# \d alunos_disciplinas;
          Tabela "public.alunos_disciplinas"
Coluna | Tipo | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id_aluno | integer |           | not null |
 id_disciplina | integer |           | not null |
 nota     | real    |           |          |
Índices:
 "alunos_disciplinas_pkey" PRIMARY KEY, btree (id_aluno, id_disciplina)
Restrições de chave estrangeira:
 "alunos_disciplinas_id_aluno_fkey" FOREIGN KEY (id_aluno) REFERENCES alunos(id)
 "alunos_disciplinas_id_disciplina_fkey" FOREIGN KEY (id_disciplina) REFERENCES disciplinas(id)

```

Note que na tabela **alunos_disciplinas** os campos **id_aluno** e **id_disciplina** são individualmente chaves estrangeiras e juntos uma chave primária composta. Foi colocado mais um atributo (Nota) do relacionamento como exemplo. Ao inserir registros, tomar os mesmos cuidados com as restrições de chaves primária e estrangeira:

1-0N / -0 → *Grav a 2ª c/ chave estrangeira e chave primária composta 56/19*

1-01 / -0 → *Grav a 2ª c/ chave primária composta e mesma da 1ª*

N-0N

importante observar.

1) Cria a 2ª como a primeira e
 cria uma 3ª para relacioná-las.

```

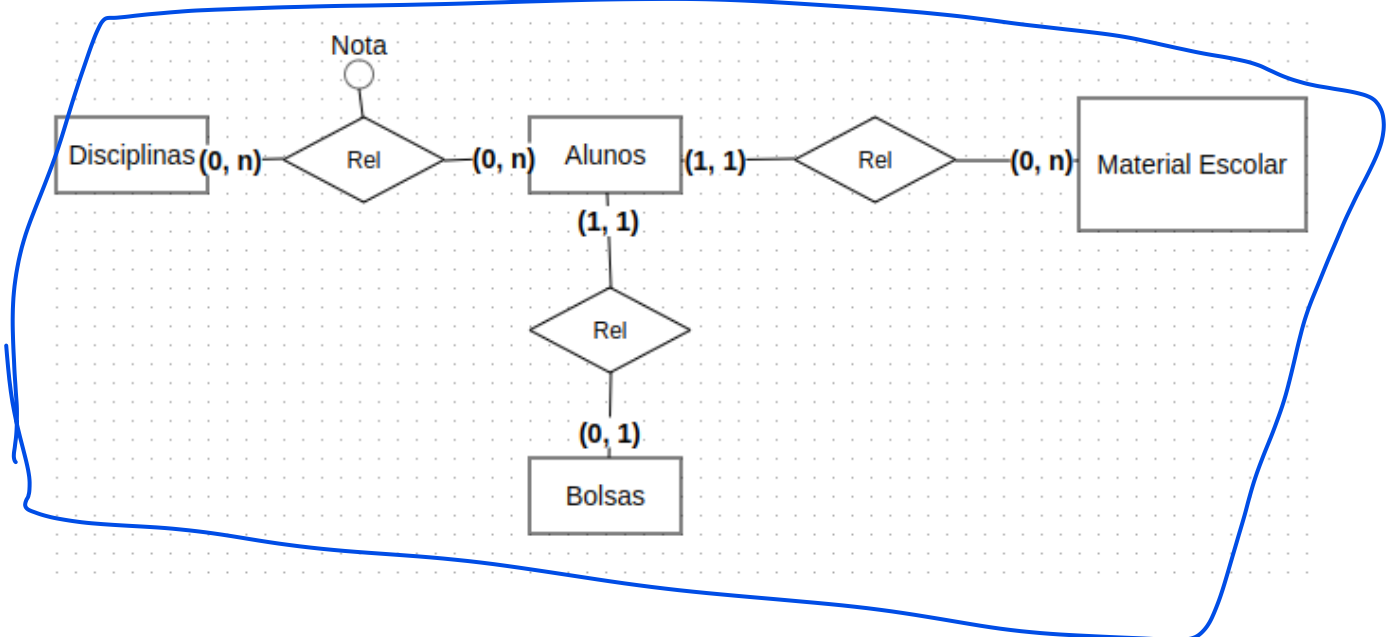
colegio=# SELECT * FROM alunos;
id | nome | idade
---+---+---
 1 | João |   29
 2 | Maria |   30
 3 | Tereza |   25
 5 | Higo |   31
(4 registros)

colegio=# SELECT * FROM disciplinas;
id | nome | ch
---+---+---
 1 | matemática | 60
 2 | português | 90
 3 | geografia | 30
(3 registros)

colegio=# SELECT * FROM alunos_disciplinas;
id_aluno | id_disciplina | nota
---+---+---
      1 |      1 | 9.5
      1 |      2 | 8
      1 |      3 | 7
      2 |      1 | 9
      2 |      2 | 6
      2 |      3 | 7.4
(6 registros)
  
```

Referência as
 2 e Diz que
 as obs são
 sobre primeira

De acordo com os registros, a interpretação que temos é que o aluno de id igual a 1 (o João) cursa as disciplinas de ids 1, 2 e 3 (matemática, português e geografia, respectivamente). E percebe que uma disciplina, por exemplo, a de id 3 (geografia), possui dois alunos de ids 1 e 2 (João e Maria, respectivamente). A seguir o esquema conceitual completos destes exemplos seguidos na apostila:



16. Cláusulas, comandos, operadores e funções SQL:

- (a) Com a cláusula **ORDER BY**, pode-se obter consultas ordenadas por uma ou mais colunas como critério. No primeiro exemplo, obtêm-se os alunos ordenados em *ordem crescente* (opcional usar o **ASC**) pela idade. No segundo, a *ordem crescente* pelo nome (alfabética) e depois pela idade. No terceiro, o uso do **DESC** para *ordem decrescente* na idade:

```
colegio=# SELECT nome, idade FROM alunos ORDER BY idade;
 nome | idade 
-----+-----
Tereza |    25
Chico Neto |    25
Maria  |    29
João   |    29
Maria  |    30
Higo   |    31
Paulo  |    50
Chico  |    77
Francisca |    77
(9 registros)
```

```
colegio=# SELECT nome, idade FROM alunos ORDER BY nome, idade;
 nome | idade 
-----+-----
Chico |    77
Chico Neto |    25
Francisca |    77
Higo   |    31
João   |    29
Maria  |    29
Maria  |    30
Paulo  |    50
Tereza |    25
(9 registros)
```

```
colegio=# SELECT nome, idade FROM alunos ORDER BY idade DESC;
 nome | idade 
-----+-----
Chico |    77
Francisca |    77
Paulo  |    50
Higo   |    31
Maria  |    30
João   |    29
Maria  |    29
Tereza |    25
Chico Neto |    25
(9 registros)
```

(b) O operador **BETWEEN** permite estabelecer um intervalo entre números, datas e textos:

```
colégio=# SELECT nome, idade FROM alunos WHERE idade BETWEEN 29 AND 50;
nome | idade
-----+-----
João  |    29
Maria |    30
Higo  |    31
Maria |    29
Paulo |    50
(5 registros)
```

```
colégio=# SELECT nome, idade FROM alunos WHERE idade NOT BETWEEN 29 AND 50;
nome | idade
-----+-----
Tereza |    25
Chico  |    77
Francisca |    77
Chico Neto |    25
(4 registros)
```

```
colégio=# SELECT nome FROM alunos WHERE nome BETWEEN 'João' AND 'Paulo';
nome
-----
João
Maria
Maria
Paulo
(4 registros)
```

(c) O comando **IN** permite estabelecer múltiplos valores numa consulta (semelhante a múltiplos OR):

```
colégio=# SELECT nome, idade FROM alunos WHERE idade IN (29, 31, 77);
nome | idade
-----+-----
João  |    29
Higo  |    31
Maria |    29
Chico  |    77
Francisca |    77
(5 registros)
```

(d) A função **COUNT()** conta registros, podendo ser estendida com critérios após a cláusula **WHERE**:

```
colégio=# SELECT * FROM alunos;
id | nome | idade
---+---+---
1 | João | 29
2 | Maria | 30
3 | Tereza | 25
5 | Higo | 31
6 | Maria | 29
7 | Paulo | 50
8 | Chico | 77
9 | Francisca | 77
10 | Chico Neto | 25
(9 registros)
```

```
colégio=# SELECT COUNT(idade) FROM alunos WHERE idade >= 50;
count
-----
3
(1 registro)
```

contando.
ou conta os alunos que tem 50 ou +

- (e) A função **SUM()** soma valores de uma coluna numérica;
- (f) Já a função **AVG()** trabalha com a média aritmética de valores numéricos;
- (g) Por sua vez, as funções **MAX()** e **MIN()**, retornam o maior e menor valor de uma coluna, respectivamente.

Veja os exemplos:

```
colégio=# SELECT * FROM alunos;
```

id	nome	idade
----	------	-------

1	João	29
2	Maria	30
3	Tereza	25
5	Higo	31
6	Maria	29
7	Paulo	50
8	Chico	77
9	Francisca	77
10	Chico Neto	25

(9 registros)

```
colégio=# SELECT SUM(idade) FROM alunos;
```

sum

373

(1 registro)

↳ Soma todas as idades das alunos.

```
colégio=# SELECT AVG(idade) FROM alunos;
```

avg

41.444444444444444444

(1 registro)

↳ MÉDIA das idades das alunos.

```
colégio=# SELECT MAX(idade) FROM alunos;
```

max

77

(1 registro)

↳ maior idade

```
colégio=# SELECT MIN(idade) FROM alunos;
```

min

25

(1 registro)

↳ menor idade



- (h) Estas funções mencionadas nos tópicos anteriores podem ser combinadas para consultas agrupadas com a cláusula **GROUP BY**. Veja exemplos em que se agrupa a contagem de idades iguais e a soma de idades por nome:

```

colegio=# SELECT * FROM alunos;
id | nome | idade
---+---+---
1 | João | 29
2 | Maria | 30
3 | Tereza | 25
5 | Higo | 31
6 | Maria | 29
7 | Paulo | 50
8 | Chico | 77
9 | Francisca | 77
10 | Chico Neto | 25
(9 registros)

colegio=# SELECT idade, COUNT(idade) FROM alunos GROUP BY idade;
idade | count
-----+-----
77 | 2
30 | 1
29 | 2
31 | 1
50 | 1
25 | 2
(6 registros)

colegio=# SELECT nome, SUM(idade) FROM alunos GROUP BY nome;
nome | sum
-----+-----
Higo | 31
Chico | 77
Maria | 59
Chico Neto | 25
Tereza | 25
Paulo | 50
João | 29
Francisca | 77
(8 registros)

```

LO conta

idades

LO Soma as idades das que têm nomes iguais

- (i) O operador **LIKE** permite buscar por determinados padrões nos valores de uma coluna. Para especificar os padrões, dois *coringas* são mais utilizados: **%** que representa zero ou mais caracteres e **_** que representa um caractere qualquer. Nos exemplos a seguir, buscamos por nomes de aluno que iniciam com Ch, terminam em o, possuem algum caractere r em qualquer posição, que contenha exatamente dois caracteres quaisquer no início do nome, sendo o r na terceira posição, respectivamente:

```

colegio=# SELECT * FROM alunos;
id | nome | idade
---+---+---
1 | João | 29
2 | Maria | 30
3 | Tereza | 25
5 | Higo | 31
6 | Maria | 29
7 | Paulo | 50
8 | Chico | 77
9 | Francisca | 77
10 | Chico Neto | 25
(9 registros)

```

```

colegio=# SELECT nome FROM alunos WHERE nome LIKE 'Ch%';
nome
-----
Chico
Chico Neto
(2 registros)

```

Ex de

Ch Carlos

Car C

Zero em qualquer posição

```

colegio=# SELECT nome FROM alunos WHERE nome LIKE '%o';
nome
-----
João
Higo
Paulo
Chico
Chico Neto
(5 registros)

```

Zero em qualquer posição e termina em o

```

colégio=# SELECT nome FROM alunos WHERE nome LIKE '%r%';
      nome
-----
Maria
Tereza
Maria
Francisca
(4 registros)

colégio=# SELECT nome FROM alunos WHERE nome LIKE '___r%';
      nome
-----
Maria
Tereza
Maria
(3 registros)

```

↳ Zero ou mais } an, ry, ...
 ⊕
 ↳ Zero ou 1 } em qualq. lugar
 ⊕
 ↳ 2 antes do r e depois Zero ou 1 ⊕

- (j) E por fim, o comando **AS** permite renomear (de forma temporária) uma coluna ou tabela com um apelido, para facilitar o acesso a blocos de consultas na query. No exemplo a seguir, realiza-se uma consulta em que retorna os três alunos com mais idade. Note que foi utilizado o comando **AS** para renomear as colunas (que aparecem no resultado) e também a própria tabela *alunos* como *a*:

```

colégio=# SELECT * FROM alunos;
 id | nome | idade
-----+-----+-----
  1 | João |   29
  2 | Maria |   30
  3 | Tereza |   25
  5 | Higo |   31
  6 | Maria |   29
  7 | Paulo |   50
  8 | Chico |   77
  9 | Francisca |   77
 10 | Chico Neto |   25
(9 registros)

colégio=# SELECT a.nome AS experientes, a.idade AS maiores_idades FROM alunos AS a ORDER BY maiores_idades DESC LIMIT 3;
 experientes | maiores_idades
-----+-----
Francisca | 77
Chico | 77
Paulo | 50
(3 registros)

```

me do (renomeado) os 3 maiores.

↳ 3
 ↳ Limita em 3

Também foi utilizado uma das cláusulas que limitam o número de registros de uma consulta. Para saber mais sobre **LIMIT** e outras cláusulas que limitam o número registros, consulte em: https://www.w3schools.com/sql/sql_top.asp

17. Foi demonstrado em aula o conceito e uso de **Subqueries** em SQL que possibilitam otimizar as consultas, por exemplo. Basicamente uma **subquery** é uma **query** dentro de outra, situada dentro parênteses. Veja um exemplo onde lista-se os nomes dos alunos de mais idade. Note a **subquery** entre parênteses sendo utilizada para buscar a maior idade que será utilizada depois na comparação da cláusula **WHERE**:

```
colegio=# SELECT * FROM alunos;
id | nome | idade
---+---+---
1 | João | 29
2 | Maria | 30
3 | Tereza | 25
5 | Higo | 31
6 | Maria | 29
7 | Paulo | 50
8 | Chico | 77
9 | Francisca | 77
10 | Chico Neto | 25
(9 registros)
```

subqueries

```
colegio=# SELECT nome FROM alunos WHERE idade = (SELECT MAX(idade) FROM alunos);
nome
-----
Chico
Francisca
(2 registros)
```

As **subqueries** podem também ser utilizadas em outras partes das consultas e também com **UPDATE**, **DELETE** e **INSERT**. Para saber mais, consulte em:

<https://www.freecodecamp.org/news/sql-subquery-how-to-sub-query-in-select-statement/>

<https://www.hashtagtreinamentos.com/otimizar-consultas-no-sql>

18. **Junções**: para consultas que envolvam mais de uma tabela, pode-se trabalhar com **JOINS** em SQL. Esta operação é similar à operação **Junção** da Álgebra Relacional, onde combina-se cada registro de uma tabela com os registros da outra tabela, no caso de apenas duas tabelas envolvidas, de acordo com um critério envolvido. Vejamos exemplos para cinco tipos de **JOINS**:

5 tipos.

```
colegio=# SELECT * FROM alunos;
id | nome | idade
---+---+---
1 | João | 29
2 | Maria | 30
3 | Tereza | 25
5 | Higo | 31
6 | Maria | 29
7 | Paulo | 50
8 | Chico | 77
9 | Francisca | 77
10 | Chico Neto | 25
(9 registros)
```

```
colegio=# SELECT * FROM material_escolar;
id | descricao | valor | id_aluno
---+---+---+---
1 | caneta azul | R$ 1,56 | 1
2 | caneta vermelha | R$ 1,90 | 1
3 | caneta preta | R$ 1,50 | 2
5 | borracha | R$ 2,90 | 3
6 | caderno | R$ 9,90 | 3
7 | estojo | R$ 5,90 | 3
9 | lápis | R$ 1,90 | 10
(7 registros)
```

- (a) **INNER JOIN**: esta junção combina somente os registros de ambas as tabelas que estão de acordo (*matching*) com o critério estabelecido após o operador *ON*. No exemplo a seguir, o resultado são alunos que possuem material escolar:

```
colégio=# SELECT nome, descricao, valor FROM alunos INNER JOIN material_escolar ON alunos.id = material_escolar.id_aluno;
 nome | descricao | valor
-----+-----+-----
 João | caneta azul | R$ 1,56
 João | caneta vermelha | R$ 1,90
 Maria | caneta preta | R$ 1,50
 Tereza | borracha | R$ 2,90
 Tereza | caderno | R$ 9,90
 Tereza | estojo | R$ 5,90
 Chico Neto | lápis | R$ 1,90
(7 registros)
```

- (b) **LEFT JOIN**: esta junção faz semelhante ao *INNER JOIN*, porém prioriza em exibir todos os registros da tabela à esquerda (no nosso exemplo, a tabela *alunos*), mesmo que fique de fora do critério. No exemplo a seguir, são listados todos os alunos, mesmo aqueles que não atendem ao critério, ou seja, que não possuem material escolar:

```
colégio=# SELECT nome, descricao, valor FROM alunos LEFT JOIN material_escolar ON alunos.id = material_escolar.id_aluno;
 nome | descricao | valor
-----+-----+-----
 João | caneta azul | R$ 1,56
 João | caneta vermelha | R$ 1,90
 Maria | caneta preta | R$ 1,50
 Tereza | borracha | R$ 2,90
 Tereza | caderno | R$ 9,90
 Tereza | estojo | R$ 5,90
 Chico Neto | lápis | R$ 1,90
 Chico |  | 
 Maria |  | 
 Paulo |  | 
 Higo |  | 
 Francisca |  | 
(12 registros)
```

- (c) **RIGHT JOIN**: dessa vez será priorizado a tabela que estiver à direita (*material_escolar*), podendo exibir material escolar que não possui aluno. Mas no nosso exemplo, devido à cardinalidade mínima de 1 para *material_escolar* e relação à *alunos*, ou seja, o material escolar já deve ser associado ao aluno, não teremos exemplos de material escolar sem aluno, sendo esse resultado semelhante ao *INNER JOIN*:

```
colégio=# SELECT nome, descricao, valor FROM alunos RIGHT JOIN material_escolar ON alunos.id = material_escolar.id_aluno;
 nome | descricao | valor
-----+-----+-----
 João | caneta azul | R$ 1,56
 João | caneta vermelha | R$ 1,90
 Maria | caneta preta | R$ 1,50
 Tereza | borracha | R$ 2,90
 Tereza | caderno | R$ 9,90
 Tereza | estojo | R$ 5,90
 Chico Neto | lápis | R$ 1,90
(7 registros)
```

- (d) **FULL JOIN**: esta junção realiza simultaneamente as três junções já citadas (*INNER*, *LEFT* e *RIGHT*). O resultado em nosso exemplo foi semelhante ao resultado do *LEFT JOIN*, dado que não temos material sem estar associado a aluno:

```
colégio=# SELECT nome, descricao, valor FROM alunos FULL JOIN material_escolar ON alunos.id = material_escolar.id_aluno;
 nome | descricao | valor
-----+-----+-----
 João | caneta azul | R$ 1,56
 João | caneta vermelha | R$ 1,90
 Maria | caneta preta | R$ 1,50
 Tereza | borracha | R$ 2,90
 Tereza | caderno | R$ 9,90
 Tereza | estojo | R$ 5,90
 Chico Neto | lápis | R$ 1,90
 Chico |  | 
 Maria |  | 
 Paulo |  | 
 Higo |  | 
 Francisca |  | 
(12 registros)
```

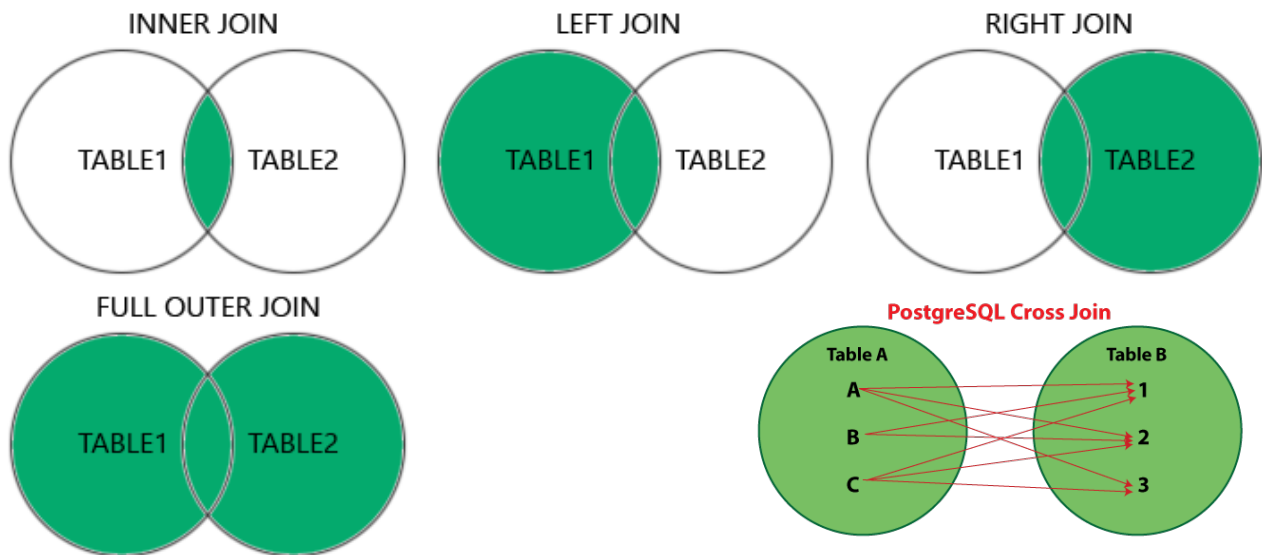
- (e) E por último, o **CROSS JOIN**, que por sua vez realiza a combinação de cada registro de uma tabela à direita com cada registro de uma tabela à esquerda, sem critérios. O resultado será então uma tabela com a multiplicação das quantidades de registros de ambas as tabelas. Para este exemplo, foram utilizada as tabelas *disciplinas* e *professores*. Assim a tabela resultante fica com 12 registros (3*4). Note também o uso do alias AS apenas para melhor apresentar os nomes das colunas no resultado:

```
colegio=# SELECT * FROM disciplinas;
id | nome      | ch
---+-----+---
 1 | matemática | 60
 2 | português | 90
 3 | geografia  | 30
(3 registros)

colegio=# SELECT * FROM professores;
id | nome  | titulacao | id_disc
---+-----+-----+-----
 1 | Beatriz | mestre    |      1
 2 | Carla  | mestre    |      1
 3 | Paulo  | especialista |     2
 4 | Rubens | doutor     |     3
(4 registros)

colegio=# SELECT disciplinas.nome AS Disciplina, ch AS CH, professores.nome AS Professor FROM disciplinas CROSS JOIN professores;
disciplina | ch | professor
---+---+-----
matemática | 60 | Beatriz
matemática | 60 | Carla
matemática | 60 | Paulo
matemática | 60 | Rubens
português  | 90 | Beatriz
português  | 90 | Carla
português  | 90 | Paulo
português  | 90 | Rubens
geografia  | 30 | Beatriz
geografia  | 30 | Carla
geografia  | 30 | Paulo
geografia  | 30 | Rubens
(12 registros)
```

E para ilustrar melhor cada um dos *JOINS* apresentados, segue uma figura que mostra a correlação com a noção de conjuntos:



Para saber mais sobre JOINS, consulte em:

<https://www.alura.com.br/artigos/join-em-sql>

https://www.w3schools.com/sql/sql_join.asp

Referências:

- <https://www.postgresql.org/>
- https://www.youtube.com/watch?v=Z_SPrzIT4Fc&list=PLucm8g_ezqNoAkYKXN_zWupyH6hQCAwxY
- <https://www.w3schools.com/sql/>